

COM3031 Probabilistic Machine Learning: Week 2

Term 2, 2025-2026

Dr Zeyu Fu, Lecturer in Computer Vision
and Machine Learning

z.fu@exeter.ac.uk

In this week

Regression

- Linear regression and Bayesian linear regression
- Generalised linear models for non-Gaussian data

Classification

- Generative and discriminative modelling approaches
- Logistic regression for binary classification

We will focus on probabilistic interpretations throughout

Part I : Regression

Linear Models

A **linear model** is one of the most widely used statistical and machine learning tools.

Linear models are often used to study relationships such as:

- income and happiness
- education and earnings
- (for us!) model complexity and predictive uncertainty

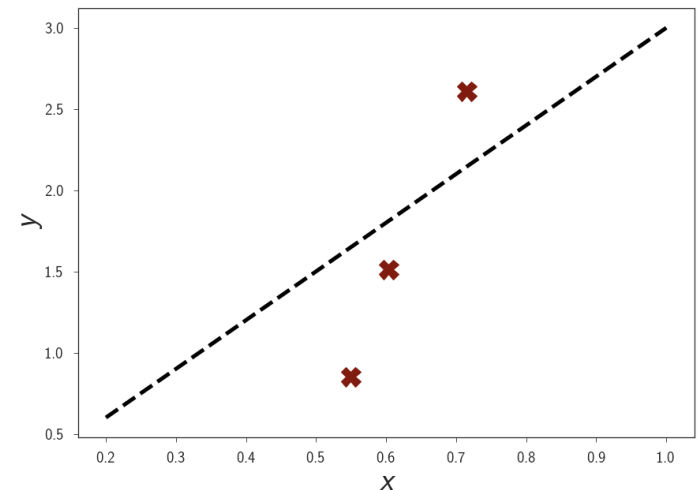
Despite their simplicity, linear models are the foundation for many more complex models used in machine learning.

Linear Regression

Simple linear regression ($D=1$, no intercept or bias):

$$y = wx + \epsilon, \text{ where } \epsilon \sim N(0, \sigma^2)$$

- y : response (output) variable
- x : explanatory (input) variable
- w : regression coefficient (slope of the line)
- ϵ : random noise capturing unmodelled effects



The dashed line shows the mean prediction; points deviate due to noise.

Key assumption:

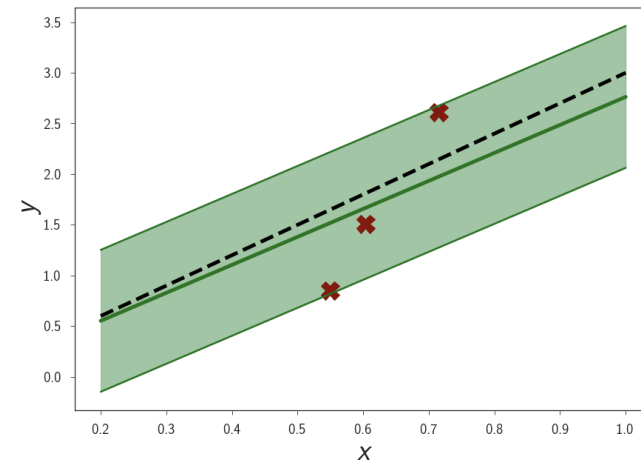
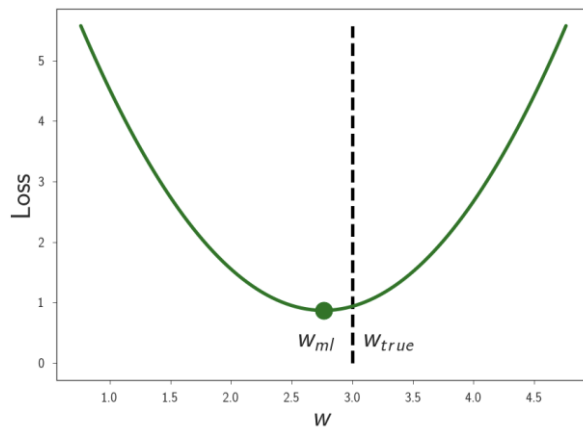
- For a fixed x , the output y is normally distributed around the line wx .

Linear Regression

Maximum likelihood estimation for simple linear regression - on board

$$p(y_i|x_i, w, \sigma^2) \sim \mathcal{N}(wx_i, \sigma^2)$$

- This means **given** x_i , the output y_i is normally distributed around the line $w x_i$.
- The likelihood measures how well a given value of w explains the observed data.
- Choose w that maximises the likelihood.



Multiple Linear Regression

- $y_i = \mathbf{w}^T \mathbf{x}_i + \epsilon$, where $\epsilon \sim \mathcal{N}(0, \sigma^2)$
 - $\mathbf{x} = (x_1, x_2, \dots, x_D)$: input features
 - $\mathbf{w} = (w_1, w_2, \dots, w_D)$: regression coefficients
 - E squared, coefficient w_j controls the influence of feature x_j
 - ϵ : Gaussian noise
- Example: House price prediction

Each weight measures the effect of one feature while holding others fixed

$$\text{Price} = w_1(\text{Size}) + w_2(\text{Bedrooms}) + w_3(\text{Localition}) + \epsilon$$

- Likelihood for the full dataset:

$$p(\mathbf{y}|\mathbf{X}, \mathbf{w}, \sigma^2) = \prod_{i=1}^n \mathcal{N}(y_i | \mathbf{w}^T \mathbf{x}_i, \sigma^2) \text{ with i.i.d assumption}$$

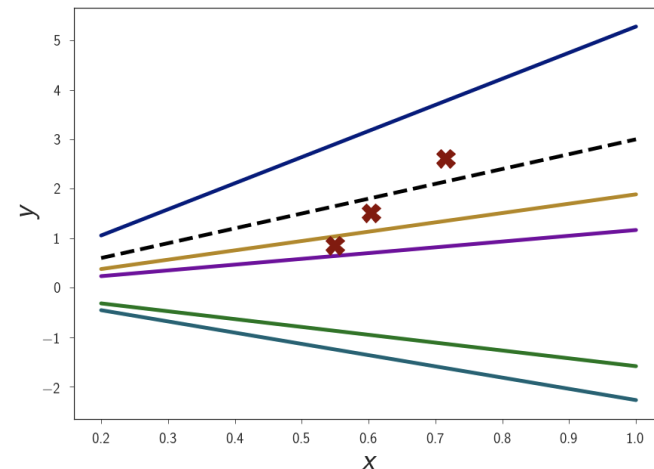
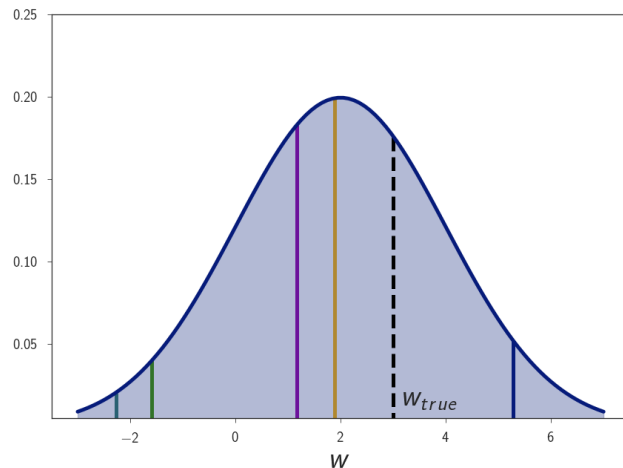
- MLE is used to find the parameters $\mathbf{w}$ (and σ^2) that best explain the observed data

Bayesian Linear Regression

- Goal: instead of a single best $\mathbf{w}$, infer a distribution over $\mathbf{w}$

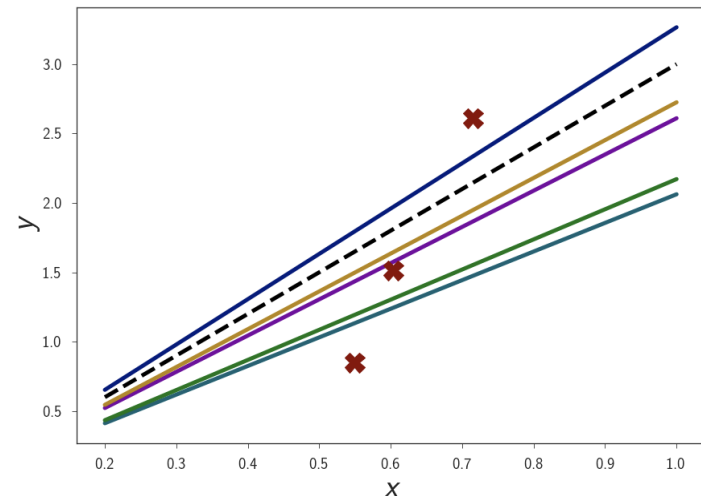
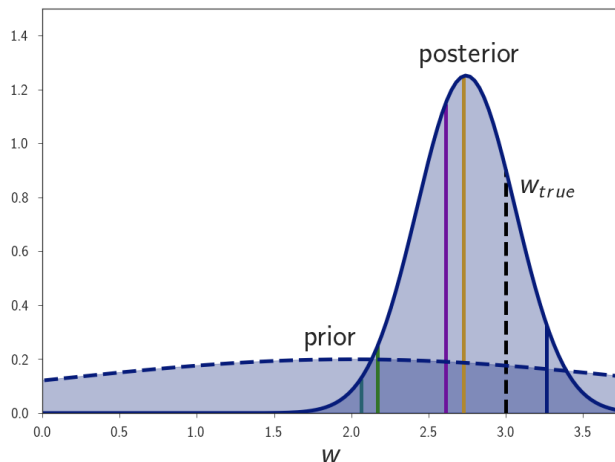
$$\mathbf{y} = \mathbf{X}\mathbf{w} + \boldsymbol{\epsilon}, \boldsymbol{\epsilon} \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$$

- $\mathbf{X} \in \mathbb{R}^{n \times d}$: design matrix (rows are inputs $\mathbf{x}_i^T$)
- $\mathbf{y} \in \mathbb{R}^n$: outputs
- $\mathbf{w} \in \mathbb{R}^d$ weights (unknown)
- Prior: $p(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{w}_0, \boldsymbol{\Sigma}_0)$ encodes beliefs about plausible weight values)



Bayesian Linear Regression

- Likelihood (data given weights): $p(\mathbf{y}|\mathbf{X}, \sigma^2, \mathbf{w}) = \mathcal{N}(\mathbf{y}|\mathbf{X}\mathbf{w}, \sigma^2 \mathbf{I})$
where $\mathbf{X}$ denotes the design matrix, $\mathbf{X} \in \mathcal{R}^{n \times d}$
- Posterior (updated beliefs after seeing data): $p(\mathbf{w}|\mathbf{y}, \mathbf{X}, \sigma^2) = \mathcal{N}(\mathbf{w}|\mathbf{w}_n, \Sigma_n)$
- Key benefit: predictions come with **uncertainty** (not just a single line).



Bayesian Linear Regression

Link to Regularisation

Using Bayes rule for Gaussians,

$$p(\mathbf{w}|\mathbf{y}, X) \propto p(\mathbf{y}|X, \mathbf{w})p(\mathbf{w})$$

- Taking logs turns products into sums:

$$\log p(\mathbf{w}|\mathbf{y}, X) = \log p(\mathbf{y}|X, \mathbf{w}) + \log p(\mathbf{w}) + \text{constant}$$

- Maximising the posterior (MAP) is equivalent to minimising:

$$\underbrace{\sum_{i=1}^n (y_i - \mathbf{w}^\top \mathbf{x}_i)^2}_{\text{data fit}}; + \underbrace{\frac{\lambda |\mathbf{w}|_2^2}{2}}_{\text{prior / regularisation}}$$

Interpretation:

- Likelihood term $\boxtimes$ fit the data
- Prior term $\boxtimes$ prefer “small” / simple weights $\boxtimes$ reduces overfitting

Generalised Linear Models

Linear regression assumes:

$$y_i = wx_i + \epsilon_i, \text{ where } \epsilon_i = \mathcal{N}(0, \sigma^2)$$

$$\text{or } y_i = \mathcal{N}(wx_i, \sigma^2)$$

- So y is continuous and errors are Gaussian

When this is not appropriate

- y is binary (0/1)
- y is a count (0,1,2,...)
- y is non-negative and skewed
- Variance changes with the mean

What GLMs change

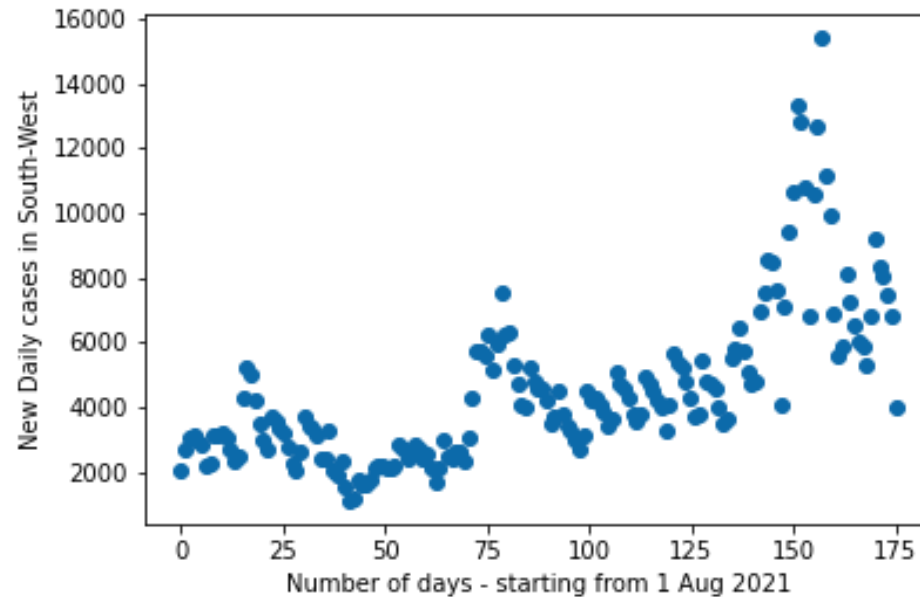
- Keep the linear predictor: wx_i
- Change the output distribution
- Use a **link function** to map predictions to a valid range

Examples

- Poisson regression $\boxtimes$ count data
- Logistic regression $\boxtimes$ binary data

A Real Example

Data source: <https://coronavirus.data.gov.uk/>



Daily coronavirus cases in South-West from 1st of Aug 2021 to 22nd Jan 2022

- Question: How do we exploit the correlation between number of cases (y) and number of days (x)?

Poisson regression

- A linear model of y_i number of COVID cases, and the number of days x_i can be written as: $y_i = wx_i + \epsilon_i$
- The linear regression is not suitable,
 - Gaussian noise can predict **negative values**
 - Counts are not well modelled by a normal distribution
 - Constant variance assumption is violated
- An appropriate probability distribution in this case is a Poisson distribution:

$$p(y_i | \lambda_i) = \frac{\lambda_i^{y_i} e^{-\lambda_i}}{y_i!}, \quad y_i \in \{0, 1, 2, \dots\}$$

- The parameter $\lambda_i > 0$ is the expected number of cases
- In Poisson distribution, the mean (or expectation) and variance is also λ .

Poisson regression

For count data, we model **expected count** $\lambda_i = E[y_i | x_i]$ with $\lambda_i > 0$

Problem with a direct linear model: $\lambda_i = wx_i$

- λ_i must be positive, but wx_i can be negative.

Solution: use a link function (GLM idea)

- Linear predictor : $\eta_i = wx_i$
- **Log link** (standard for Poisson): $\log(\lambda_i) = \eta_i$
 - The log link connects the mean of a Poisson distribution to a linear model
- Equivalently: $\lambda_i = \exp(wx_i)$, guarantees $\lambda_i > 0$
 - In ML terms, $\exp(\cdot)$ plays a role similar to an *activation function*

Link Functions

Distribution	Mean (μ)	Link function (η)	Link name
Normal	$\mu \in (-\infty, +\infty)$	μ	unit link
Gamma	$\frac{\alpha}{\beta} \in (0, +\infty)$	$\frac{1}{\mu}$	Negative inverse
Poisson	$\lambda \in (0, +\infty)$	$\log(\mu)$	log
Binomial	$np \in (0, n)$	$\log \frac{\mu}{n-\mu}$	logit
Bernoulli	$p \in (0, 1)$	$\log \frac{\mu}{1-\mu}$	logit

Link functions of commonly used distributions

MLE for Poisson regression

- Given independence across observations: $p(\mathbf{y} | X, \mathbf{w}) = \prod_{i=1}^n \frac{\lambda_i^{y_i} e^{-\lambda_i}}{y_i!}$

Substitute $\lambda_i = \exp(\mathbf{w}^\top \mathbf{x}_i)$,

- MLE is easier using the log-likelihood:

$$\begin{aligned} \widehat{\mathbf{w}}_{\text{MLE}} &= \arg \max_{\mathbf{w}} \ell(\mathbf{w}) \quad \ell(\mathbf{w}) = \sum_{i=1}^n [y_i \mathbf{w}^\top \mathbf{x}_i - \exp(\mathbf{w}^\top \mathbf{x}_i) - \log(y_i!)] \\ &= \log p(\mathbf{y} | X, \mathbf{w}) \quad \boxtimes \text{ objective function} \end{aligned}$$

- There is **no closed-form solution** (unlike linear regression). Instead, we use **iterative optimisation**, such as: Gradient ascent, Newton–Raphson and Iteratively Reweighted Least Squares (IRLS).

Summary

Linear regression:

- Assumes Gaussian noise, constant variance, and continuous outputs
- Not suitable for binary or count data

Generalised Linear Models (GLMs):

- Extend linear models to non-Gaussian data
- Still use a linear predictor

Key differences from linear regression:

- The response y can follow non-Gaussian distributions (e.g. Poisson, Bernoulli)
- Errors do not need to be normally distributed
- Variance can depend on the mean

Link function connects the **mean of the distribution** to the linear predictor and

Can be **non-linear** (e.g. log link in Poisson regression)

Part II : Classification

Linear Classification

Goal: predict a class label $y \in \{0,1\}$ (or $\{1, \dots, K\}$) from features x

Discriminative models (direct):

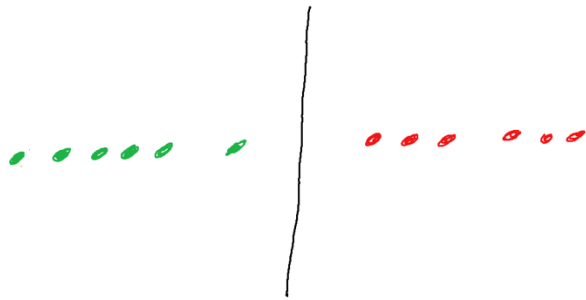
- learn the conditional distribution of the output data $p(y | x)$, and predict by choosing the most likely class: $\hat{y} = \arg \max_y p(y | x)$
- Example: Logistic regression

Generative models (model the data):

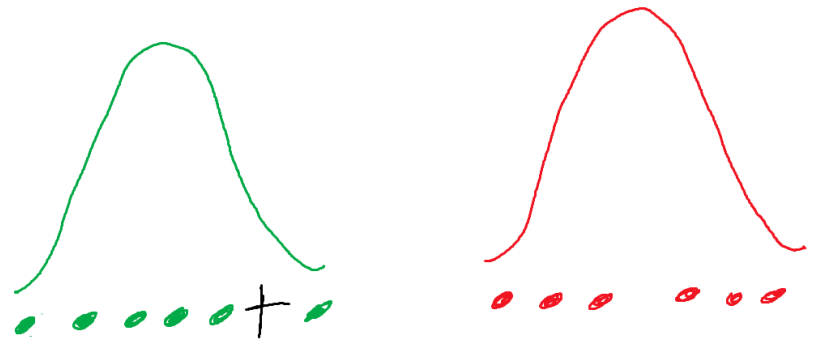
- learn the joint distribution over the inputs and output $p(x, y) = p(x | y)p(y)$
- Use Bayes' rule to get $p(y | x)$
- Example: Naive Bayes

Generative and Discriminative Models: Intuition

- A binary classification problem (e.g. *rain vs no rain*, *survival vs non-survival*)



Discriminative: focuses on the *decision boundary* (“separate classes well”)



Generative: models how each class *generates data* (“what does class data look like?”)

Generative and Discriminative Models: Intuition

Task: Determine the language someone is speaking (e.g. English vs French)

Generative model:

- Learns what each language sounds like
- Models how speech features are generated **given the language**
- Chooses the language that best explains the observed speech
- Example idea: ***“Does this audio look more like English or French?”***

Discriminative model:

- Learns the differences between languages directly
- Does **not** model how speech is generated
- Focuses on predicting the language label from features
- Example idea: ***“Given this audio, which language is it?”***

Discriminative Models

Inputs and labels: $\mathcal{D} = (X, y)$

Goal: Learn a model that directly predicts the class label y from features x

Model: Learn the conditional distribution $p(y | x, \theta)$. This models *how likely each class is given the input*.

Learning (training): Choose parameters θ to maximise the conditional likelihood:

$$\theta = \operatorname{argmax}_{\theta} \prod_{i=1}^n p(y_i | x_i, \theta)$$

- **Prediction:** Once θ is known, we can predict the class of a new point x' by computing : $p(y' | x', \theta)$ and $\hat{y} = \operatorname{argmax}_y p(y | x', \theta)$

Bernoulli model (No Features)

Binary outcome: $y_i \in \{0,1\}$. Examples: rain / no rain, pass / fail

Assumption: each observation is a Bernoulli trial with the same probability θ

- The likelihood is: $p(y | \theta) = \prod_{i=1}^n \theta^{y_i} (1 - \theta)^{1-y_i}$
- After maximising the likelihood function: $\hat{\theta} = \frac{1}{n} \sum_{i=1}^n y_i$

Interpretation:

- The MLE is simply the **proportion of class 1** in the data
- **No input features** x ☐ same probability for everyone

Limitation:

- Cannot adapt predictions to different inputs

From Bernoulli Model to Logistic Regression

Key idea: Replace the constant probability $p(y_i = 1) = \theta$ with an input dependent probability. $\theta_i = p(y_i = 1 | x_i)$

- Try a linear model (a natural first attempt): $\theta_i = w_0 + w_1 x_i$
 - Problem: probabilities must satisfy $0 \leq \theta_i \leq 1$. Linear functions are unbounded.
- Transform the probability
 - A quantity can take any real value $(-\infty, +\infty)$ and still maps back to a valid probability in $[0,1]$.
 - Use log-odds (logit): $\log \frac{\theta_i}{1 - \theta_i} = w_0 + w_1 x_i$ (unbounded)
 - This is the key modelling step in logistic regression.

Logistic Regression

- Linear predictor (score):

$$\eta_i = w_0 + w_1 x_i \quad (\text{or } \eta_i = w^\top x_i)$$

- Link function (logit):

$$\text{logit}(\theta_i) = \log \frac{\theta_i}{1 - \theta_i} = \eta_i$$

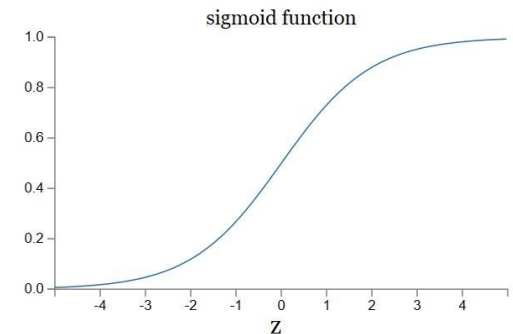
- Inverse link (sigmoid) gives a valid probability:

$$\theta_i = \sigma(\eta_i) = \frac{1}{1 + \exp(-\eta_i)}$$

- Prediction (simple rule):

$$\hat{y}_i = 1 \text{ if } \theta_i \geq 0.5, \text{ else } 0$$

- Threshold can be adjusted depending on the application



Logistic Regression

Likelihood: Each observation follows a **Bernoulli distribution**:

Assuming independence across data points:

$$p(y \mid X, w) = \prod_{i=1}^n \theta_i^{y_i} (1 - \theta_i)^{1-y_i}$$

Log-likelihood: We maximise the **log-likelihood** for numerical stability:

$$\ell(w) = \sum_{i=1}^n [y_i \log \theta_i + (1 - y_i) \log(1 - \theta_i)]$$

MLE: $\hat{w} = \arg \max_w \ell(w)$

- There is no closed-form solution
- We use **iterative optimisation** (e.g. gradient ascent / descent)

From Logistic Regression to a Bayesian View

Key takeaway of logistic regression

- **Training:** Maximise the log-likelihood and it is equivalent to minimising cross-entropy loss
- **Parameters:** Treated as fixed but unknown and the result is a single best estimate (or one best decision boundary)
- No explicit measure of **uncertainty** in the parameters or predictions

What if we want to know **how confident** we are in the learned weights and predictions?

Bayesian Logistic Regression

Key idea: Treat the weights as **random variables**, not fixed values

Likelihood (same as logistic regression):

$$y_i \sim \text{Bernoulli}(\theta_i), \quad \theta_i = \sigma(w^\top x_i)$$

Prior on weights: $w \sim \mathcal{N}(w_0, \Sigma_0)$

Posterior: $p(w | D) \propto p(y | X, w), p(w)$, where $\mathcal{D}$ is the dataset

Note:

- There is no closed form for the posterior
- Posterior needs to be approximated - Markov Chain Monte Carlo, Laplace approximation and Variational inference.

Generative Models

Key idea: Instead of modelling the conditional probability $p(\mathbf{y} | \mathbf{X})$ directly, generative models learn the **joint distribution** of data and labels.

- **Bayes rule** (for classification):
 - **class-conditional distribution** (how features look given a class)
 - $p(\mathbf{y})$: **class prior** (how common each class is)
 - $p(\mathbf{X})$: **normalising constant** (same for all classes)
- **Data setup**
 - $\mathbf{X} \in \mathbb{R}^{n \times D}$: design matrix (features)
 - $\mathbf{y} = (y_1, \dots, y_n)^T$: class labels
 - Classification labels: $y_i \in \{0, 1, \dots, K\}$

Generative Models

Learning in a generative model

- Learn $p(X | y = k)$ for each class k
- Learn the class prior $p(y = k)$
- Use Bayes' rule to compute $p(y | X)$

Examples of generative classifiers

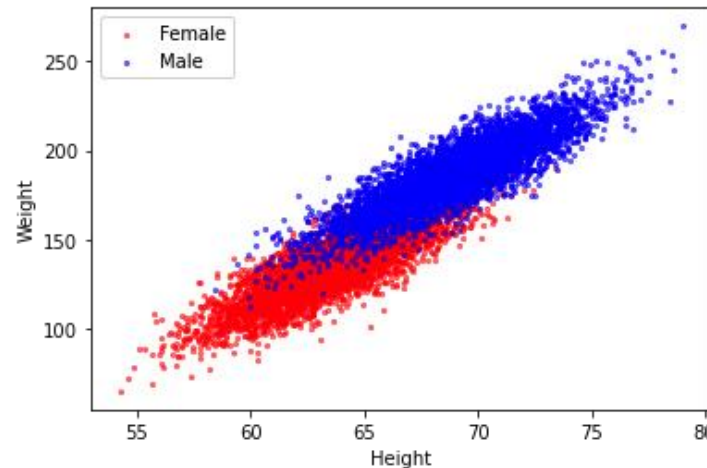
- Naive Bayes
- Gaussian Discriminant Analysis

Why use generative models?

- Can work well with small datasets
- Can generate new samples
- Naturally fit into the Bayesian framework

Generative Classifier

Task: We want to predict the class $y \in \{\text{Female}, \text{Male}\}$ from features $x = (\text{Height}, \text{Weight})$.



Source: <https://www.kaggle.com/mustafaali96/weight-height>

Idea: learn how each class “generates” features:

$p(x | y = k)$ what height-weight data looks like for each class

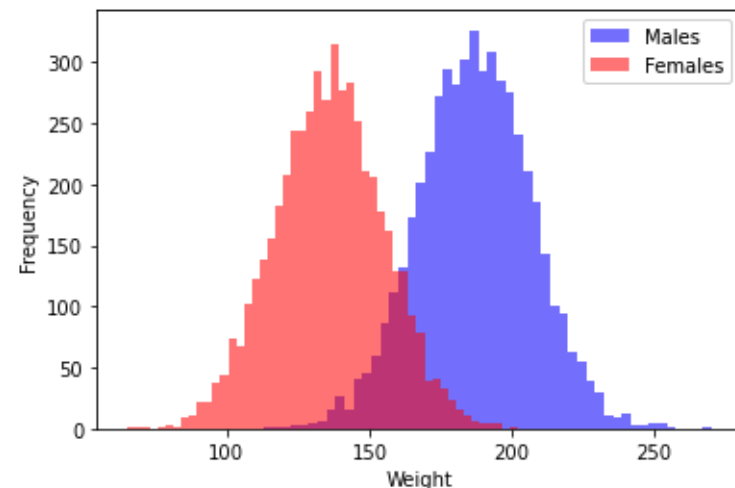
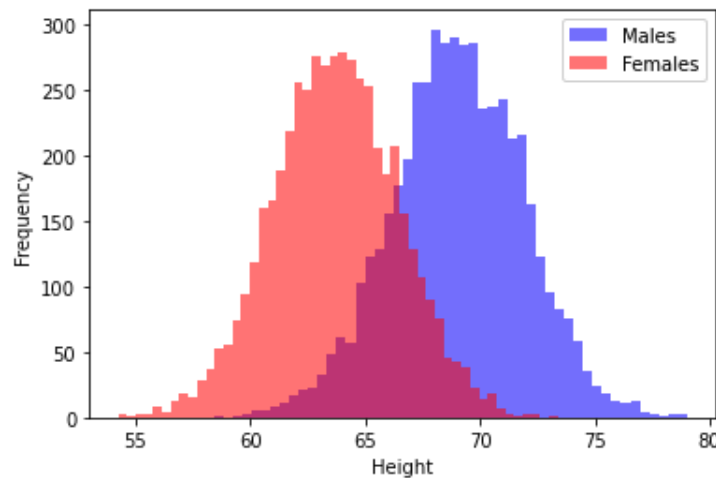
and $p(y = k)$ (how common each class is)

Then classify using Bayes’ rule:

$$p(y = k | x) \propto p(x | y = k)p(y = k)$$

Generative Classifier

- *What does $p(x | y)$ actually look like in real data?*



Distribution of height (left) and weight (right) in two classes

- Left plot: $p(\text{Height} | y = \text{Male})$, $p(\text{Height} | y = \text{Female})$
- Right plot: $p(\text{Weight} | y = \text{Male})$, $p(\text{Weight} | y = \text{Female})$

Naive Bayes

Naive Bayes assumption (the “naive” part)

- We have features $\mathbf{x} = (x_1, \dots, x_D)$ and class $y \in \{1, \dots, K\}$
- Conditional independence given the class:

$$p(\mathbf{x} \mid y = k) = \prod_{j=1}^D p(x_j \mid y = k)$$

- For Height–Weight ($D=2$):

$$p(\text{Height, Weight} \mid y = k) \approx p(\text{Height} \mid y = k)p(\text{Weight} \mid y = k)$$

- This is a simplifying assumption (often not exactly true), but it can still work well.

Prediction rule

$$\hat{y} = \arg \max_k \left[\log p(y = k) + \sum_{j=1}^D \log p(x_j \mid y = k) \right]$$

- Using logs turn products into sums and avoid tiny numbers.

Gaussian Naive Bayes

Model choice for continuous features

- Assume each feature is Gaussian within each class:

$$p(x_j | y = k) = \mathcal{N}(x_j | \mu_{jk}, \sigma_{jk}^2)$$

- For height-weight ($x = (h, w)$):

$$p(h | y = k) = \mathcal{N}(h | \mu_{h,k}, \sigma_{h,k}^2), \quad p(w | y = k) = \mathcal{N}(w | \mu_{w,k}, \sigma_{w,k}^2)$$

What we estimate from data (per class k)

- Prior: $\pi_k = p(y = k)$
- Means and variances: $\mu_{h,k}, \sigma_{h,k}^2, \mu_{w,k}, \sigma_{w,k}^2$ (*just sample mean/variance within each class*)

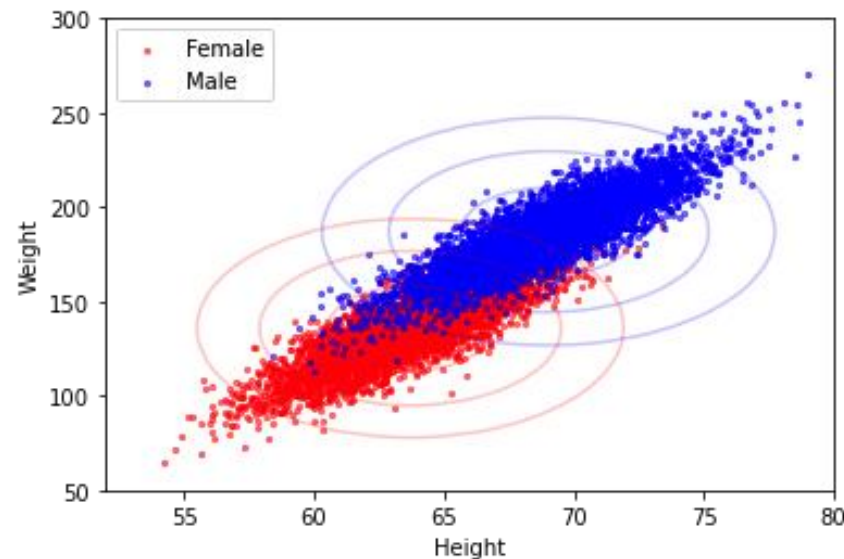
Classification rule

$$\hat{y} = \arg \max_k [\log \pi_k + \log \mathcal{N}(h | \mu_{h,k}, \sigma_{h,k}^2) + \log \mathcal{N}(w | \mu_{w,k}, \sigma_{w,k}^2)]$$

Decision Boundaries

How the decision boundary appears

- Boundary is where the two class scores are equal.
- With Gaussians, the boundary is **often curved (quadratic)** (not necessarily a straight line).
- It “emerges” from the fitted class distributions.



The decision boundary lies roughly where the red and blue contours intersect.

Naive Bayes Variants

Which Naive Bayes should we use? (depends on what “x” looks like)

- Gaussian Naive Bayes (continuous features)
 - $x_j \in R$: real-valued measurements
 - Example: height & weight (what we just fitted $\boxtimes$ decision boundary)
- Bernoulli Naive Bayes (binary features)
 - $x_j \in \{0,1\}$: word *present* / *not present*
 - Example: spam detection using “contains *free*?” “contains *win*?”
- Multinomial Naive Bayes (count features)
 - $x_j \in \{0,1,2, \dots\}$: word counts / bag-of-words
 - Example: topic classification using word frequencies

Summary

Generative models

- Learn $p(x | y)$ and $p(y)$
- Classify using Bayes: $p(y | x) \propto p(x | y)p(y)$
- Good when: small data, missing features, want to *simulate / generate* data
- Examples: Naive Bayes, Gaussian Discriminant Analysis (LDA/QDA), Mixture Models, Hidden Markov Models (sequence models)

Discriminative models

- Learn $p(y | x)$ or a decision boundary directly
- Good when: lots of labelled data, best predictive accuracy is the goal
- Examples: Logistic regression, Support Vector Machines, k-Nearest Neighbours, Neural Networks
- *(Structured outputs: Conditional Random Fields)*